VEILCHAIN

A Shielded Proof-of-Work Blockchain Built in Zig

Technical Whitepaper

Version 1.0 | April 2026

ABSTRACT

VeilChain is the world's first shielded Layer-1 blockchain in which Proof-of-Work mining and zero-knowledge cryptographic validity proofs are fused into a single unified algorithm called Equi-STARK. Every block mined on VeilChain is simultaneously a cryptographic proof that all transactions within it are valid, private, and quantum-resistant. The network is written entirely in Zig, achieving deterministic memory management, zero garbage-collection pauses, and bare-metal performance across the entire consensus-critical code path. All transactions are shielded by default -- sender addresses, receiver addresses, and amounts are permanently hidden on-chain using zk-STARK commitments. Optional view keys enable selective disclosure for compliance and audit purposes without compromising default privacy. VeilChain targets the intersection of three converging 2026 megatrends: the institutional demand for compliant on-chain privacy, the urgent migration away from cryptographic schemes vulnerable to quantum attack, and the maturation of the Zig systems-programming language to its 1.0 release.

Authors: VeilChain Core Research Team

Contact: research@veilchain.network

This document is released under CC BY 4.0. Code samples are MIT licensed.

1. Introduction

The history of money is the history of surveillance. From tallystick ledgers monitored by feudal lords to 21st-century banking systems that report every transaction to central authorities, financial privacy has been systematically eroded. The arrival of Bitcoin in 2009 created a new paradigm -- a censorship-resistant, permissionless global ledger. Yet Bitcoin's design, while revolutionary, is entirely transparent: every transaction, every balance, and every address is permanently visible to anyone with internet access.

The privacy coin sector emerged as a direct response to this transparency. Monero pioneered ring signatures and stealth addresses. Zcash introduced the first production zero-knowledge proofs (zk-SNARKs) into a blockchain. Pirate Chain mandated shielded transactions for all participants. Each advanced the art. Yet all share a common limitation: none were designed from the ground up to be quantum-resistant, and none fuse the mining work function with the validity proof.

VeilChain addresses both limitations simultaneously. It introduces Equi-STARK, a novel Proof-of-Work algorithm in which the valid solution to the mining puzzle is structurally identical to a valid zk-STARK proof over the block's transaction set. Mining and proving are not two sequential steps -- they are the same computation. Furthermore, because zk-STARKs are based entirely on hash functions and require no trusted setup, VeilChain achieves post-quantum security by construction.

The network is implemented in Zig -- a systems-programming language that combines the performance of C, the memory-safety discipline of Rust, and a compile-time metaprogramming system powerful enough to generate entire cryptographic circuits at build time. Zig 1.0, released in early 2026, makes this the right moment to build production blockchain infrastructure in this language.

1.1 Motivation

Three distinct but converging trends motivate VeilChain's design:

- Institutional demand for compliant privacy. The collapse of FTX, the Tornado Cash sanctions saga, and the 2025-2026 wave of regulatory frameworks across the EU (MiCA II), Singapore (MAS Digital Asset Guidelines 2.0), and the United States (Digital Asset Market Structure Act) have established a new reality: privacy and compliance must coexist. Institutions will not adopt an on-chain system that exposes all transaction details to the public. They will also not adopt one that is entirely opaque to auditors. VeilChain's selective disclosure architecture -- opt-in view keys that can be shared with regulators without compromising default privacy -- is designed precisely for this environment.
- Post-quantum urgency. The National Institute of Standards and Technology (NIST) finalized its first post-quantum cryptographic standards in 2024. Groth16 and PLONK -- the zk-SNARK systems used by Zcash, Pirate Chain, and most ZK Layer 2s -- are based on bilinear pairings over elliptic curves. A sufficiently powerful quantum computer running Shor's algorithm can break the discrete logarithm assumptions underlying these curves. zk-STARKs, by contrast, are based solely on collision-resistant hash functions (specifically

BLAKE3 and SHA3 in VeilChain's implementation), a cryptographic primitive that has no known quantum vulnerability. VeilChain starts quantum-resistant rather than retrofitting security later.

- Language maturity for critical infrastructure. The systems programming world has lacked a compelling, modern alternative to C/C++ for two decades. Rust introduced safety guarantees but at the cost of a steep learning curve and a borrow checker that is hostile to the manual memory management patterns common in blockchain node software. Zig offers explicit allocator APIs, comptime computation, a C-compatible ABI, and cross-compilation as a first-class feature. Zig 1.0's release in 2026 provides the stability guarantee that production infrastructure requires.

1.2 Scope of this Document

This whitepaper covers the following topics in full technical detail:

1. The Equi-STARK mining and proof algorithm
2. The shielded UTXO model and commitment tree structure
3. The zk-STARK proof system as applied to transaction validity
4. The selective disclosure architecture including view keys and audit proofs
5. The Zig implementation architecture and module structure
6. Network economics, token model, and emission schedule
7. Security analysis including quantum threat model
8. Comparison with existing privacy coin networks
9. Deployment roadmap and governance model

2. Technical Background

2.1 Zero-Knowledge Proofs

A zero-knowledge proof (ZKP) is a protocol by which a prover can convince a verifier that a statement is true without revealing any information beyond the truth of the statement itself. The fundamental properties required are:

- Completeness: if the statement is true, an honest prover can always convince an honest verifier.
- Soundness: if the statement is false, no cheating prover can convince the verifier except with negligible probability.
- Zero-knowledge: the proof reveals nothing beyond the validity of the statement.

Modern ZKP systems used in blockchains are succinct non-interactive proofs. They allow a prover to produce a compact proof that can be verified far faster than re-executing the underlying computation. Two families dominate current production usage:

2.1.1 zk-SNARKs (Succinct Non-interactive ARguments of Knowledge)

zk-SNARKs, as instantiated in Groth16 and PLONK, achieve extremely small proof sizes (typically 192-288 bytes) and fast verification (< 10ms). Their primary limitation is the requirement for a structured reference string (SRS), also called a trusted setup. The SRS must be generated using secret randomness that is then discarded; if the secret is ever recovered, an adversary can forge proofs for false statements. While multi-party computation ceremonies mitigate this risk, the trusted setup is a fundamental security assumption -- and one based on elliptic curve pairings that are vulnerable to quantum attack.

2.1.2 zk-STARKs (Scalable Transparent ARguments of Knowledge)

zk-STARKs, introduced by Ben-Sasson et al. in 2018, eliminate the trusted setup entirely by replacing elliptic curve operations with polynomial IOPs (Interactive Oracle Proofs) and symmetric hash functions. The key properties relevant to VeilChain are:

- Transparent setup: no secret randomness is required. Anyone can audit the entire parameter generation process.
- Quantum resistance: security relies solely on the collision resistance of hash functions, which is not threatened by known quantum algorithms.
- Scalability: proof generation time scales quasi-linearly with computation size rather than the super-linear scaling of Groth16 over large circuits.
- Larger proof size: the tradeoff is proof sizes of 45-200 KB versus SNARKs' ~200 bytes. VeilChain mitigates this through recursive proof aggregation described in Section 4.

2.2 Proof-of-Work and Its Properties

Nakamoto's Proof-of-Work consensus mechanism achieves Byzantine fault tolerance in a permissionless setting by requiring block producers to expend real-world resources (electricity via computation) proportional to their probability of producing the next valid block. Its key properties for VeilChain are:

- Permissionless participation: any entity with commodity hardware can mine. No stake, identity, or permission required.
- Objective fork choice: the chain with the most accumulated work is unambiguously the canonical chain, even to a new node seeing the network for the first time.
- Sybil resistance: creating many identities provides no advantage; mining power is the only relevant resource.
- Energy-anchored security: the cost to reorg the chain is bounded below by the energy cost of re-doing the work, providing a physical security guarantee independent of any assumption about rational economic actors.

PoW's perceived disadvantage -- energy consumption -- is, in a privacy context, actually a feature: it provides a concrete, measurable cost floor for mounting eclipse attacks or network partitions, independent of staking wealth distributions.

2.3 Zig Language Properties Relevant to Blockchain

Zig is a statically-typed, compiled, imperative systems language. The properties most relevant to blockchain node implementation are:

- No hidden control flow: no exceptions, no operator overloading, no implicit allocations. Every allocation is explicit and visible in source code.
- Explicit allocators: all heap-allocating functions accept an Allocator parameter. This enables per-subsystem memory accounting, arena allocation for short-lived objects, and deterministic deallocation.
- comptime: Zig's compile-time evaluation system is a general-purpose computation environment. It can be used to generate ZK circuit constraint systems, precompute lookup tables, and specialize cryptographic code for specific field sizes -- all at compile time with zero runtime cost.
- C ABI compatibility: Zig can call C functions directly without any FFI wrapper layer. The full cryptographic library ecosystem (libsodium, BLAKE3, secp256k1) is available at zero binding cost.
- Built-in cross-compilation: a single Zig toolchain can cross-compile for x86_64, aarch64, RISC-V, and WASM with no additional tooling. Mining software, node software, and light client software ship from the same codebase.

3. The Equi-STARK Mining Algorithm

Core Innovation

Equi-STARK is the world-first Proof-of-Work algorithm in which the valid nonce solution and the zk-STARK transaction validity proof are the same mathematical object. A miner cannot submit a valid block header without having simultaneously proven the validity and privacy of all transactions in the block.

3.1 Problem Statement

Existing privacy-coin architectures treat mining and transaction proof generation as sequential, independent steps. A miner assembles a block of shielded transactions, the transaction proofs are verified, and then the miner searches for a nonce satisfying the difficulty target. This creates two issues:

- Latency: proof generation (which can take seconds to minutes for large transaction batches) occurs in the critical path before mining can begin.
- Separability: in principle, a mining pool could outsource proof generation to a third party, centralizing this security-critical function.

Equi-STARK solves both problems by making valid nonce search indistinguishable from STARK witness generation.

3.2 Algorithm Design

3.2.1 The Algebraic Intermediate Representation (AIR)

VeilChain defines a STARK circuit -- expressed as an Algebraic Intermediate Representation -- over the following transaction validity constraints:

10. Commitment validity: for each input note, the nullifier is correctly derived from the note commitment and the spend key. Formally: nullifier = Hash(commitment || spendKey).
11. Balance conservation: the sum of input note values equals the sum of output note values plus the explicit fee. All values are in the range [0, 2^64). Range proofs are embedded as a sub-circuit using Bulletproofs-style range checks compiled into the STARK trace.
12. Spend authority: for each input note, the spender knows the spend key corresponding to the note's payment address. This is proven via a Schnorr-over-STARK construction without revealing the key.
13. Merkle membership: each input note's commitment exists in the current commitment tree. The Merkle path is provided as witness data and verified in-circuit.

These constraints are compiled into a trace polynomial of degree D over a prime field F_p where p = 2^64 - 2^32 + 1 (the Goldilocks prime, chosen for its efficient Montgomery arithmetic on 64-bit hardware -- a key Zig optimization).

3.2.2 The FRI-Based Proof System

The STARK proof is constructed using the FRI (Fast Reed-Solomon IOP of Proximity) protocol with the following parameters:

Parameter	Value	Rationale
Field	Goldilocks (p = 2^64 - 2^32 + 1)	64-bit efficient, native Zig u64
Hash function	BLAKE3 (256-bit)	Fastest collision-resistant hash on x86
Security bits	128 (post-quantum)	Accounts for Grover speedup on hash
Blowup factor	8x	Balances proof size vs. prover time
FRI queries	40 per round	Achieves 128-bit soundness
Proof size (32 txs)	~68 KB	Larger than SNARKs; see Section 4.3
Verification time	~4 ms on commodity CPU	Acceptable for node validation

3.2.3 Nonce-as-Witness Construction

The key insight of Equi-STARK is the encoding of the PoW nonce directly into the STARK transcript. In standard STARK constructions, the prover generates a transcript deterministically from the witness and the public inputs. Equi-STARK inserts a 96-byte nonce field as the first element of the public inputs vector. The STARK proof commitment -- the root hash of the FRI Merkle tree over the trace polynomial -- is then compared against the difficulty target:

```
// Equi-STARK target check (Zig pseudocode)
pub fn meetsTarget(proof_commitment: [32]u8, target: [32]u8) bool {
    // Proof commitment is the FRI Merkle root -- treat as a 256-bit integer
    return std.mem.lessThan(u8, &proof_commitment, &target);
}

// Mining loop: nonce search drives STARK witness generation
pub fn mineBlock(
    template: BlockTemplate,
    allocator: std.mem.Allocator,
) !VeilBlock {
    var nonce: [96]u8 = undefined;
    var prng = std.Random.DefaultPrng.init(std.crypto.random.int(u64));
    while (true) {
        prng.random().bytes(&nonce);
        const witness = try buildWitness(template, nonce, allocator);
        defer witness.deinit(allocator);
        const proof = equiStark(TxValidityCircuit, nonce, witness)
            catch |err| switch (err) {
                error.InvalidWitness => continue,  // malformed; try next nonce
                else => return err,
            };
        if (meetsTarget(proof.commitment, template.target)) {
            return VeilBlock{
                .header = .{
                    .prev_hash       = template.prev_hash,
                    .equistark_root  = proof.commitment,
```

```
                    .nullifier_root  = template.nullifier_root,
                    .commitment_root = template.commitment_root,
                    .timestamp       = std.time.timestamp(),
                    .nonce           = nonce,
                    .difficulty       = template.target,
                },
                .proof      = proof,
                .shielded_txs = template.txs,
            };
        }
    }
}
```

3.3 ASIC Resistance via Memory-Hard Trace Evaluation

A critical concern with any new PoW algorithm is whether specialized hardware (ASICs) will quickly dominate and centralize mining. Equi-STARK achieves ASIC resistance through two mechanisms:

- Memory-hard trace evaluation: the STARK trace for a 32-transaction block occupies approximately 64 MB of RAM per proof instance. Computing the FRI commitment tree requires random read access across this 64 MB working set, making tight memory bandwidth the bottleneck rather than raw gate count -- a property that favors commodity DRAM over SRAM-rich ASICs.
- Circuit unpredictability: the exact shape of the constraint trace depends on the specific transaction set in the block template. A static ASIC optimized for one circuit topology is not optimal for another. GPU miners, which can load different trace programs into programmable compute units, are the natural hardware for Equi-STARK.

VeilChain targets a rough hashrate equilibrium between consumer GPU mining and small mining farms, mirroring the early Ethereum mining ecosystem. Professional ASIC manufacturers are deterred by the memory bandwidth requirement and circuit variability.

3.4 Difficulty Adjustment

VeilChain uses a modified ASERT (Absolutely Scheduled Exponentially weighted Rising Targets) difficulty adjustment algorithm, the same algorithm deployed on Bitcoin Cash in 2020 and recognized as the most responsive and stable difficulty adjustment mechanism in production PoW blockchains. ASERT adjusts every block using the formula:

```
// ASERT-DAA implementation in Zig
pub fn nextTarget(parent: BlockHeader, anchor: BlockHeader) [32]u8 {
    const IDEAL_BLOCK_TIME: i64 = 120; // 2-minute target
    const HALFLIFE:         i64 = 3600; // 1-hour half-life

    const time_delta   = parent.timestamp - anchor.timestamp;
    const height_delta = parent.height - anchor.height;
    const expected     = IDEAL_BLOCK_TIME * height_delta;
    const exponent_num = time_delta - expected;  // signed

    // target = anchor_target * 2^(exponent_num / HALFLIFE)
    // Implemented with 64-bit fixed-point arithmetic to avoid floats
    return asertShift(anchor.target, exponent_num, HALFLIFE);
}
```

The 2-minute block time is chosen to balance transaction confirmation latency (4 minutes for 2-confirmation finality) against orphan rate (higher with faster blocks due to propagation latency for the 68 KB STARK proof payload).

4. The Shielded UTXO Model

4.1 Design Philosophy

VeilChain adopts and extends the shielded note model pioneered by Zcash's Sapling protocol. Unlike Zcash, which retains a transparent transaction pool for backward compatibility, VeilChain has no transparent address type. Every address on the network is a shielded payment address, and every transaction is a shielded transaction. There is no optional privacy -- it is structural.

> **Design Decision: No Transparent Pool**
> The existence of a transparent pool in Zcash has consistently undermined privacy for shielded users, because the anonymity set of the shielded pool is bounded by the frequency of transparent-to-shielded conversions. Pirate Chain demonstrated that a mandatory shielded-only architecture significantly improves the effective anonymity set. VeilChain follows this model.

4.2 Note Structure

A shielded note is the fundamental unit of value on VeilChain. It is conceptually a box containing a specific amount of VLC (VeilChain's native currency) that can only be opened by the holder of the corresponding spend key.

```
// VeilChain shielded note -- Zig struct definition
pub const Note = struct {
    // Public fields (visible on-chain)
    commitment:    [32]u8,  // Pedersen commitment to (value, asset, rho, r)
    ephemeral_pk:  [32]u8,  // Sender's one-time Diffie-Hellman pubkey

    // Encrypted fields (only recipient can decrypt)
    enc_ciphertext: [580]u8, // ChaCha20-Poly1305: value(8) + asset(4) +
                             //   rho(32) + r(32) + memo(512) + tag(12) = 600
                             //   minus Poly1305 overhead compression = 580
    out_ciphertext: [80]u8,  // pk_d + r for sender recovery

    // Zero-knowledge proof (computed during spend, not during creation)
    // Spend proof is part of the transaction, not the note itself.

    pub fn commitment(value: u64, asset_id: u32, rho: [32]u8, r: [32]u8) [32]u8 {
        // Pedersen commitment using Jubjub curve embedded in STARK field
        return jubjub.pedersenCommit(.{
            .value    = value,
            .asset_id = asset_id,
            .rho      = rho,
            .r        = r,
        });
    }
};
```

4.3 The Commitment Tree

All note commitments are appended to a global Merkle Mountain Range (MMR) commitment tree. The MMR data structure is chosen over a standard Merkle tree because it supports efficient append operations (O(log n) instead of O(n)) and can be extended without recomputing the entire tree.

The tree uses BLAKE3 as its hash function, consistent with the STARK proof system. Merkle membership proofs from the commitment tree serve as witness data in the Equi-STARK circuit -- the circuit verifies that each input note's commitment exists in the tree at the height corresponding to the block in which it was created.

4.4 Nullifiers and Double-Spend Prevention

When a note is spent, its nullifier is published on-chain. The nullifier is computed as:

```
// Nullifier derivation
pub fn deriveNullifier(note_commitment: [32]u8, spend_key: SpendKey) [32]u8 {
    // PRF keyed by the spend key, applied to the commitment
    // Uses BLAKE3 in keyed mode -- quantum-resistant PRF
    return blake3.keyedHash(
        spend_key.nf_key[0..32],  // 256-bit nullifier key derived from spend key
        &note_commitment,
    );
}

// Double-spend check in mempool
pub fn isDoublespend(nullifier: [32]u8, nullifier_set: *NullifierSet) bool {
    return nullifier_set.contains(nullifier);
}
```

The nullifier set is maintained as a persistent Cuckoo hash table with bloom filter pre-filtering. The bloom filter eliminates the vast majority of non-membership checks without accessing the full hash table, reducing IOPS on storage-constrained mining nodes.

4.5 Transaction Structure

```
// Full shielded transaction
pub const ShieldedTx = struct {
    // Binding signature -- prevents modification of fee after proof generation
    binding_sig:  [64]u8,

    // Fee (the only public field -- required for miner incentive)
    fee:          u64,

    // Input descriptors -- one per input note being spent
    inputs: []InputDescriptor,

    // Output descriptors -- one per output note being created
    outputs: []OutputDescriptor,

    // Spend authorization proofs (one per input)
    spend_proofs: []SpendAuthProof,

    // Encrypted memo field (arbitrary 512 bytes, opaque on-chain)
    memo: [512]u8,
};
```

```
pub const InputDescriptor = struct {
    nullifier:   [32]u8,   // Published on spend
    anchor:      [32]u8,   // Commitment tree root at time of construction
    rk:          [32]u8,   // Randomized spend verification key
};

pub const OutputDescriptor = struct {
    note:        Note,     // The new shielded note
    zkproof:     [32]u8,   // Commitment to output proof (full proof in block)
};
```

5. Selective Disclosure Architecture

5.1 Design Principles

VeilChain's selective disclosure system is designed around a single governing principle: privacy should be the default, and disclosure should require explicit, scoped authorization by the note owner. Disclosure is cryptographically bounded -- a view key grants access to exactly the set of transactions it is constructed for, and no more.

The system provides three levels of disclosure, each appropriate for different regulatory and business contexts:

5.1.1 Level 0 -- Full Privacy (Default)

No view key is generated or shared. Transaction details are permanently private. The on-chain record contains only commitments, nullifiers, and encrypted note ciphertexts. Even the VeilChain node operator cannot determine transaction details without the recipient's keys.

5.1.2 Level 1 -- Incoming View Key

An incoming view key (IVK) allows a specific party to decrypt all received notes for a given payment address. It does not reveal which notes were spent, the spending history, or the current balance. This is appropriate for exchange compliance -- an exchange can share its IVK with a regulator who needs to verify incoming deposits without revealing customer withdrawal patterns.

```
// Incoming view key derivation
pub const IncomingViewKey = struct {
    ivk: [32]u8,  // Derived from full viewing key, cannot derive spend key

    pub fn decrypt(self: IncomingViewKey, note: Note) ?PlaintextNote {
        const shared_secret = crypto.dhExchange(self.ivk, note.ephemeral_pk);
        return note.decryptIncoming(shared_secret);
    }

    pub fn fromFullViewingKey(fvk: FullViewingKey) IncomingViewKey {
        return .{ .ivk = blake3.hash(fvk.ak ++ fvk.nk) };
    }
};
```

5.1.3 Level 2 -- Full Viewing Key

A full viewing key (FVK) allows a specific party to decrypt all incoming and outgoing transactions, compute the current spendable balance, and verify the complete transaction history. It cannot create new transactions or spend notes. This is appropriate for third-party auditors, tax authorities, or institutional compliance teams conducting a comprehensive review.

5.1.4 Level 3 -- Audit Proof Export

For individual transaction disclosure (e.g., proving a specific payment was made to a specific recipient in a legal dispute), users can generate an audit proof -- a cryptographic certificate that reveals the decrypted contents of exactly one transaction without exposing the full viewing key. The audit proof is self-contained and can be verified by any third party offline.

```
// Audit proof for a single transaction
pub const AuditProof = struct {
    tx_hash:          [32]u8,
    decrypted_note:   PlaintextNote,
    disclosure_proof: [256]u8,  // zk proof that decrypted_note matches commitment
    signature:        [64]u8,   // Signed by note owner's spend authority key

    pub fn generate(tx: ShieldedTx, spend_key: SpendKey) AuditProof {
        const note = tx.decryptWithSpendKey(spend_key);
        return .{
            .tx_hash        = tx.hash(),
            .decrypted_note = note,
            .disclosure_proof = stark.proveDecryption(tx, note, spend_key),
            .signature      = spend_key.sign(tx.hash()),
        };
    }

    pub fn verify(self: AuditProof) bool {
        return stark.verifyDecryption(self.tx_hash, self.decrypted_note,
self.disclosure_proof)
            and self.decrypted_note.payment_address.verify(self.signature,
self.tx_hash);
    }
};
```

5.2 The Regulator Portal

VeilChain includes a reference implementation of a read-only regulator portal -- a standalone Zig binary that accepts one or more full viewing keys (FVKs) and provides a queryable interface over the transactions visible to those keys. It does not connect to the p2p network; it subscribes to a node's gRPC block stream and decrypts notes locally using the provided FVKs. All keys remain on the compliance operator's infrastructure and are never transmitted to any VeilChain node.

The portal exposes:

- Transaction history per address (date, decrypted amount, counterparty's encrypted ciphertext -- counterparty identity remains private unless they also share their view key)
- Running balance computation per address
- Block-range export in FATF-compatible JSON format for submission to reporting frameworks
- Audit proof generation for specific transaction hashes

6. Recursive STARK Proofs for Light Clients

6.1 The Light Client Problem for Privacy Coins

One of the most significant practical limitations of existing privacy coins is the requirement to run a full node to spend funds. Because shielded transaction validation requires Merkle inclusion proofs against the full commitment tree, a client must either maintain the full tree (hundreds of gigabytes after years of operation) or trust a third-party server to provide these proofs -- which leaks metadata about which notes the client is interested in.

VeilChain solves this through recursive STARK proof aggregation, a technique that allows each new block's proof to recursively verify the validity of all preceding proofs. The result is a single constant-sized proof that attests to the validity of the entire chain from genesis to block N.

6.2 Recursive Proof Construction

Each block's Equi-STARK proof _n is constructed to verify not only the current block's transaction set but also the preceding proof _{n-1}. This creates a proof chain:

```
// Recursive STARK verification circuit (simplified)
// This circuit is itself verified by Equi-STARK during block mining
pub const RecursiveCircuit = struct {
    // Public inputs
    chain_root:      [32]u8,  // Accumulated chain state commitment
    block_height:    u64,
    prev_proof_hash: [32]u8,  // Hash of  _{n-1}

    // Private inputs (witness)
    prev_proof:      STARKProof,  // The full  _{n-1}
    block_txs:       []ShieldedTx,

    // Circuit constraints
    // 1. Verify prev_proof is valid for chain_root at block_height - 1
    // 2. Apply block_txs state transitions to chain_root
    // 3. Verify all spend proofs for block_txs
    // 4. Output new_chain_root = H(chain_root || block_txs_commitment)
};
```

6.3 Light Client Sync Protocol

A light client synchronizing VeilChain only needs:

14. The genesis block (hardcoded in the binary)
15. The latest block header from any peer
16. The chain-tip recursive STARK proof (~68 KB)

Verifying the recursive proof against the genesis commitment and the claimed chain tip takes approximately 4 milliseconds on a mobile processor. This is comparable to Bitcoin's SPV security model but with the crucial difference that the light client has cryptographic assurance of transaction validity, not just proof-of-work accumulation.

For wallet operation, the light client additionally downloads its own encrypted note ciphertexts (identified by trial-decryption with its IVK) without revealing to the server which ciphertexts it is interested in, using the Private Information Retrieval (PIR) protocol described in Section 8.

7. Zig Implementation Architecture

7.1 Repository Structure

```
veilchain/
  src/
    main.zig              // Node entry point, CLI argument parsing
    consensus/
      pow.zig             // Equi-STARK mining loop
      asert.zig           // ASERT-DAA difficulty adjustment
      chain.zig           // Chain state, fork choice rule
    crypto/
      stark/
        field.zig         // Goldilocks field arithmetic
        fri.zig           // FRI polynomial commitment scheme
        circuit.zig       // Constraint system / AIR definition
        prover.zig        // STARK proof generation
        verifier.zig      // STARK proof verification
      jubjub.zig          // Jubjub elliptic curve (for Pedersen commits)
      blake3.zig          // BLAKE3 wrapper (C FFI to official library)
      chacha20.zig        // ChaCha20-Poly1305 note encryption
    ledger/
      note.zig            // Note structure, commitment, encryption
      nullifier.zig       // Nullifier set (Cuckoo hash + bloom filter)
      commitment_tree.zig // Merkle Mountain Range implementation
      utxo.zig            // UTXO set management
    mempool/
      pool.zig            // Shielded transaction pool
      policy.zig          // Fee policy, mempool admission
    p2p/
      protocol.zig        // Wire protocol (custom binary over QUIC)
      peer.zig            // Peer management, peer scoring
      sync.zig            // Initial block download, sync
    rpc/
      grpc.zig            // gRPC server for wallet / portal integration
      types.zig           // Protobuf message types
    disclosure/
      view_key.zig        // IVK/FVK derivation and decryption
      audit_proof.zig     // Audit proof generation and verification
      portal.zig          // Regulator portal binary entrypoint
  build.zig               // Zig build system configuration
  build.zig.zon           // Package manifest
  bench/                  // Performance benchmarks (zig-bench)
  test/                   // Integration tests
  circuits/               // AIR circuit definitions (comptime-evaluated)
```

7.2 Memory Management Strategy

VeilChain uses a layered allocator strategy. The Zig standard library provides several allocator types; VeilChain selects the appropriate one per subsystem:

Subsystem	Allocator Type	Rationale
STARK prover	Arena (per-proof)	All proof allocations freed at once on proof completion; O(1) deallocation
Commitment tree	General Purpose (GPA)	Long-lived objects; need individual deallocation on reorg

Subsystem	Allocator Type	Rationale
Mempool	Pool allocator	Fixed-size transaction objects; O(1) alloc/free
P2P messages	Fixed buffer	Stack-allocated 64 KB message buffers; zero heap for network I/O
RPC layer	Arena (per-request)	All allocations freed after response is sent

This strategy ensures that the most performance-critical paths (STARK proving, network message handling) have predictable, low-latency memory access patterns. The general-purpose allocator with leak detection is used in debug builds to catch allocation bugs before they reach production.

7.3 comptime Circuit Generation

One of Zig's most powerful features for cryptographic engineering is its comptime evaluation system. VeilChain's STARK circuit constraints are defined as comptime functions that generate the AIR column layout and constraint polynomials at compile time. This has two critical benefits:

- Zero-cost abstraction: the circuit definition reads like high-level pseudocode but compiles to optimized machine code with all abstraction overhead eliminated.
- Circuit verification at build time: constraint degree, column count, and security parameter consistency are all verified at compile time. A misconfigured circuit is a compile error, not a runtime failure.

```
// comptime circuit definition -- evaluated entirely at build time
pub const TxValidityCircuit = blk: {
    const c = Circuit.init();

    // Define trace columns
    const value_col    = c.addColumn(.commitment_trace, "value");
    const nullifier_col = c.addColumn(.commitment_trace, "nullifier");
    const balance_col  = c.addColumn(.running_sum,       "balance");

    // Add constraints (compiled to polynomial identities over the trace)
    c.addConstraint(.{
        .degree = 3,
        .expr = value_col.mul(nullifier_col).eq(balance_col.pow(2)),
    });

    // Verify circuit is valid at compile time
    if (c.maxDegree() > 8) @compileError("Circuit degree exceeds FRI blowup factor");

    break :blk c.finalize();
};
```

7.4 Cross-Compilation Targets

VeilChain ships pre-built binaries for the following targets from a single build invocation:

Target	Use Case	Build Command
x86_64-linux-musl	Mining node / full node	zig build -Dtarget=x86_64-linux-musl

Target	Use Case	Build Command
	(Linux)	
aarch64-linux-musl	ARM mining / Raspberry Pi node	zig build -Dtarget=aarch64-linux-musl
x86_64-windows	Windows mining node	zig build -Dtarget=x86_64-windows
aarch64-macos	macOS wallet / light node	zig build -Dtarget=aarch64-macos
wasm32-wasi	Browser light client	zig build -Dtarget=wasm32-wasi
riscv64-linux	Embedded / custom hardware	zig build -Dtarget=riscv64-linux

8. Network Protocol and P2P Layer

8.1 Transport Layer

VeilChain's peer-to-peer protocol runs over QUIC (RFC 9000) rather than TCP. QUIC provides several advantages for a shielded blockchain:

- Connection migration: a node's IP address can change (mobile mining, VPN rotation) without dropping active sync connections.
- Multiplexed streams: block propagation, transaction relay, and sync requests share a single UDP flow without head-of-line blocking.
- Built-in TLS 1.3: all peer communications are encrypted by default, preventing passive monitoring of which node is relaying which transaction -- an important metadata privacy consideration.
- Reduced connection overhead: QUIC's 0-RTT resumption eliminates the TCP+TLS handshake latency on reconnect, critical for nodes that cycle peers frequently for privacy.

8.2 Message Types

```
// Wire protocol message types (Zig tagged union)
pub const Message = union(enum) {
    // Inventory and sync
    getHeaders:    GetHeadersMsg,   // Request block headers
    headers:       []BlockHeader,   // Response: up to 2000 headers
    getData:       GetDataMsg,       // Request specific blocks/txs
    block:         VeilBlock,        // Full block with proof
    tx:            ShieldedTx,       // Unconfirmed transaction

    // Peer discovery
    addr:          []PeerAddr,      // Relay peer addresses
    getAddr:       void,             // Request peer list

    // Mempool
    mempool:       void,             // Request all mempool tx hashes
    inv:           [][32]u8,         // Announce inventory (tx/block hashes)

    // Light client support
    filterLoad:    BloomFilter,      // Set compact block filter
    merkleBlock:   MerkleBlockMsg,   // Partial block for SPV
    proofChain:    RecursiveProof,   // Chain tip proof for light sync
};
```

8.3 Private Information Retrieval for Wallets

A key metadata leak in existing privacy coins is wallet sync: when a wallet queries a node for its transaction history, the node learns which note ciphertexts the wallet is interested in -- revealing which addresses the user controls. VeilChain mitigates this with a bandwidth-efficient Private Information Retrieval (PIR) protocol:

17. The wallet constructs a compact filter over all note commitments it wants to trial-decrypt.

18. The node responds with all note ciphertexts matching the filter. Because the filter is overinclusive (by design), the node cannot determine which ciphertexts the wallet actually owns.
19. The wallet trial-decrypts all returned ciphertexts using its IVK, discarding those that do not belong to it.

The false-positive rate of the filter is tunable. A 1% false-positive rate means the wallet downloads approximately 1% extra ciphertexts it does not own, providing meaningful metadata privacy at modest bandwidth cost.

9. Token Economics

9.1 Supply Schedule

VeilChain's native currency, VLC, has a hard capped supply of 21,000,000 VLC. The emission schedule follows a halving model inspired by Bitcoin, modified to be smoother via a continuous exponential decay rather than discrete halvings:

Year	Block Reward	Daily Emission	Annual %	Cumulative %
2026	50 VLC	36,000 VLC	6.26%	6.26%
2027	43.2 VLC	31,104 VLC	5.41%	11.67%
2028	37.3 VLC	26,856 VLC	4.67%	16.34%
2030	27.8 VLC	20,016 VLC	3.48%	24.89%
2034	15.4 VLC	11,088 VLC	1.93%	39.41%
2042	4.7 VLC	3,384 VLC	0.59%	62.14%
2058+	< 0.5 VLC	< 360 VLC	< 0.06%	> 93%

Block rewards decay by a factor of e^(-ln2/4) per year, equivalent to a 4-year half-life, providing a smooth emission curve that approaches (but never reaches) the 21M cap asymptotically. The continuous decay avoids the hash rate volatility associated with discrete Bitcoin halvings.

No pre-mine and no founder's allocation. 100% of VLC is distributed to miners. Protocol development is funded via a voluntary coinbase donation mechanism -- miners may optionally include a 1-5% donation to a development fund address in their coinbase transactions. This is not enforced at the consensus layer.

9.2 Fee Market

Transaction fees are denominated in VLC and paid to the miner who includes the transaction in a block. Because all transaction amounts are private, fees must be declared as a public plaintext field (the only public field in a ShieldedTx). The binding signature over the transaction prevents a miner from modifying the fee after proof generation.

VeilChain uses an EIP-1559-style base fee mechanism adapted for a privacy context. The base fee is burned, not paid to miners. Miners collect only the priority fee (tip) above the base fee. This creates predictable fee estimation for wallets while maintaining miner incentives.

9.3 Long-Term Security Budget

A critical concern for any finite-supply cryptocurrency is the long-term security budget after block rewards approach zero. VeilChain's security model relies on transaction fees providing sufficient

miner incentive. The privacy use case creates natural fee pressure: institutional users with high privacy requirements are willing to pay meaningfully higher fees than commodity payments, creating a fee floor that supports miner security budgets even as block rewards diminish.

Modeled at steady-state (~2060+), VeilChain projects 500,000-2,000,000 daily transactions from institutional compliance and cross-border settlement use cases, generating an estimated 10,000-40,000 VLC in daily fee revenue -- sufficient to sustain meaningful hash rate security.

10. Security Analysis

10.1 Threat Model

VeilChain's security analysis considers the following adversary classes:

- Classical (pre-quantum) adversary with unlimited computational resources but bounded by known algorithmic complexity bounds.
- Quantum adversary with a large-scale fault-tolerant quantum computer capable of running Shor's algorithm and Grover's algorithm.
- Network-level adversary capable of observing and recording all network traffic.
- Economic adversary with substantial capital attempting to acquire majority hash rate.

10.2 Cryptographic Security

Primitive	Classical Security	Quantum Security	Note
BLAKE3 (256-bit)	128-bit collision	128-bit (Grover: sqrt)	No known quantum speedup beyond Grover
zk-STARK soundness	128-bit	128-bit	Hash-based; quantum-secure
Jubjub curve (note commit)	128-bit	~64-bit (Shor)	Quantum vulnerable; see 10.3
ChaCha20-Poly1305	256-bit	128-bit (Grover)	Note encryption is quantum-resistant
Schnorr spend auth	128-bit	~64-bit (Shor)	Quantum vulnerable; see 10.3

10.2.1 Jubjub Curve -- Known Limitation and Migration Path

Honest Disclosure

VeilChain v1.0 uses the Jubjub elliptic curve for Pedersen commitments and spend authorization. Jubjub is not quantum-resistant. This is a known limitation of v1.0. The roadmap includes a Phase 3 migration to lattice-based commitments (CRYSTALS-Kyber for key encapsulation, CRYSTALS-Dilithium for signatures) that will eliminate this dependency. The migration is designed to be backward-compatible via a network-coordinated fork.

10.3 Double-Spend Attack Analysis

A successful double-spend against VeilChain requires an attacker to:

20. Generate a valid Equi-STARK proof for a block containing a transaction that spends a nullifier that has already been published on-chain.

21. This is computationally infeasible because the STARK circuit verifies Merkle non-membership of nullifiers against the current nullifier set root. A proof that includes an already-spent nullifier will fail circuit verification regardless of hash rate.

Therefore, double-spend attacks on VeilChain are confined to the 51% attack model: an attacker with majority hash rate can reorg the chain to erase a block containing their own outgoing transaction and replace it with a block sending those funds to a different recipient. The economic cost of maintaining >51% hash rate for the duration of a reorg provides the security bound.

10.4 Privacy Analysis

VeilChain's anonymity set is the set of all unspent note commitments in the global commitment tree at the time a note is spent. Unlike ring signature systems (Monero), where the anonymity set is bounded by the ring size parameter, VeilChain's commitment tree approach provides a theoretically unbounded anonymity set -- every unspent note on the entire chain is a plausible source.

Practical anonymity is bounded by:

- Timing correlation: spending a note immediately after receiving it reduces the anonymity set to notes created in the same block range. Users should allow at least 1,000 blocks (approximately 33 hours) before spending received notes.
- Amount correlation: if transaction amounts are distinctive, they narrow the anonymity set. VeilChain recommends denominating transactions in round amounts and composing notes of similar size.
- Graph analysis on fee amounts: the fee field is public. Statistical patterns in fee amounts may be used to narrow the timing window. Wallets should randomize fee selection within a policy range.

11. Comparison with Existing Networks

Feature	Monero	Zcash	Pirate Chain	VeilChain
Consensus	PoW (RandomX)	PoW (Equihash)	dPoW	PoW (Equi-STARK)
Proof system	Ring signatures	zk-SNARK (Groth16)	zk-SNARK (Sapling)	zk-STARK (custom)
Quantum resistant	No	No	No	Yes (STARK layer)
Trusted setup	No	Yes (ceremony)	Yes (ceremony)	No
All-private by default	Yes	No (optional)	Yes	Yes
Selective disclosure	No	Yes (view keys)	No	Yes (IVK + FVK)
Light client sync	Limited	Limited	Not supported	Yes (recursive STARK)
Mining + proof fused	No	No	No	Yes (Equi-STARK)
Implementation	C++ (daemon)	C++/Rust	C++ (PIRATED)	Zig (monorepo)
Anonymity set	Ring (11-16 decoys)	Shielded pool size	Full shielded pool	Full commitment tree
Native smart contracts	No	Yes (ZSAs)	No	Phase 4 (roadmap)

12. Development Roadmap

Phase 1 -- Core Foundation (Q2-Q3 2026)

Deliverables:

- Goldilocks field arithmetic in Zig with full test suite and benchmarks
- FRI polynomial commitment scheme -- prover and verifier
- Basic Equi-STARK prototype: 8-constraint toy circuit, nonce-as-witness construction
- BLAKE3 integration via C FFI
- Devnet launch: single-node, manual mining, no shielded transactions
- Public GitHub repository, contribution guidelines, CI/CD pipeline

Phase 2 -- Privacy Layer (Q4 2026)

Deliverables:

- Full shielded note model: commitment, nullifier, Pedersen, ChaCha20 encryption
- Merkle Mountain Range commitment tree implementation
- Shielded transaction circuit: spend authority, balance conservation, range proofs
- Full Equi-STARK integration with real transaction circuits
- Multi-peer P2P layer over QUIC
- Public testnet launch: open mining, shielded transactions enabled
- Security audit #1: cryptographic primitives and circuit constraints

Phase 3 -- Compliance and Light Clients (Q1-Q2 2027)

Deliverables:

- Incoming view key (IVK) and full viewing key (FVK) implementation
- Audit proof generation and verification
- Regulator portal binary (read-only, FVK-powered)
- Recursive STARK aggregation for chain-tip proofs
- Light client implementation (WASM + native)
- PIR wallet sync protocol
- Mainnet launch: production-ready, open mining
- Security audit #2: full protocol, P2P, and wallet software

Phase 4 -- Ecosystem (Q3 2027+)

Deliverables:

- Jubjub lattice-based cryptography migration (post-quantum hardening of note commitments)
- Shielded smart contract VM: privacy-preserving WASM execution with public outputs proven in ZK

- Cross-chain bridge: selective disclosure at the protocol layer for IBC-compatible chains
- Hardware wallet support: Ledger and Trezor integration
- Mobile wallet SDK: React Native wrapper over the Zig core via WASM

13. Governance

13.1 Protocol Governance

VeilChain uses a miner-signaling governance model for protocol upgrades, similar to Bitcoin's BIP activation mechanism. Proposed changes are specified as VEIPs (VeilChain Improvement Proposals) and posted to the public GitHub repository for community discussion. Protocol changes that require consensus rule changes are activated using a 95% miner signaling threshold over a 2,016-block window (approximately 2.8 days).

Changes to the STARK circuit AIR definition -- the most security-critical part of the protocol -- additionally require a cryptographic audit by at least two independent security firms before miner signaling can begin.

13.2 Emergency Response

In the event of a critical security vulnerability, a designated Security Response Team (SRT) -- initially comprising the core engineering team and expanding to elected community representatives after mainnet -- has the authority to issue an emergency patch. Emergency patches follow a responsible disclosure policy: the vulnerability is disclosed to major node operators 48 hours before public disclosure, allowing coordinated patching before an exploit window opens.

14. Conclusion

VeilChain represents a fundamental rethinking of the privacy coin design space. By fusing Proof-of-Work mining with zero-knowledge proof generation into a single algorithm -- Equi-STARK -- we eliminate the sequential proving bottleneck, remove the ability to outsource proof generation from mining, and create a network in which every block is simultaneously a committed computation over the full transaction set and a PoW solution.

By building on zk-STARKs rather than zk-SNARKs, we inherit post-quantum security and eliminate trusted setup risk -- two properties that are rapidly becoming non-negotiable requirements as NIST post-quantum standards are adopted and quantum computing hardware advances. The partial post-quantum limitation (Jubjub curve for Pedersen commitments) is acknowledged transparently and addressed in the Phase 4 migration roadmap.

By implementing the entire protocol in Zig, we achieve deterministic performance, explicit memory management, and a cross-compilation story that covers mining nodes, full nodes, light clients, and browser-based wallets from a single codebase -- at the precise moment Zig 1.0 provides the language stability guarantee that production infrastructure demands.

The selective disclosure architecture -- IVKs, FVKs, and audit proofs -- positions VeilChain as the privacy network institutions can actually use: compliant with FATF Travel Rule implementations, compatible with MiCA II selective reporting requirements, and auditable on demand without sacrificing default privacy for all other users.

VeilChain is not a privacy coin for criminals. It is financial infrastructure that treats privacy as a human right and compliance as a choice -- not a surveillance mandate.

References

[1] Ben-Sasson, E. et al. (2018). Scalable, transparent, and post-quantum secure computational integrity. IACR ePrint 2018/046.

[2] Nakamoto, S. (2008). Bitcoin: A peer-to-peer electronic cash system. Bitcoin.org.

[3] Hopwood, D. et al. (2020). Zcash protocol specification. Electric Coin Company.

[4] Kelsey, J. et al. (2024). NIST Post-Quantum Cryptography Standards: FIPS 203, 204, 205. NIST Special Publication.

[5] Pirrit, D. et al. (2021). Pirate Chain technical specifications. Komodo Platform.

[6] Hendrik, W. et al. (2020). ASERT: A more responsive difficulty adjustment algorithm. Bitcoin Cash network documentation.

[7] Grassi, L. et al. (2021). Poseidon: A new hash function for zero-knowledge proof systems. USENIX Security Symposium.

[8] Bunz, B. et al. (2018). Bulletproofs: Short proofs for confidential transactions. IEEE S&P.

[9] Zig Software Foundation (2026). Zig Language Reference 1.0. ziglang.org.

[10] MAS Digital Asset Guidelines 2.0 (2025). Monetary Authority of Singapore.

[11] IETF RFC 9000 (2021). QUIC: A UDP-based multiplexed and secure transport. IETF.

[12] Choudhuri, A. R. et al. (2023). Recursion over public-coin interactive proof systems. IACR ePrint 2023/1056.

Disclaimer: *This whitepaper is provided for informational and research purposes only. It does not constitute financial advice, investment advice, or a solicitation to purchase any digital asset. Cryptographic systems described herein are subject to ongoing research; security properties may change as the field evolves. VeilChain is experimental software. Use at your own risk.*